AI Forensics: Reconstructing Prompt-Injection

1st Paolo Vasquez
paolo.vasquez@utec.edu.pe

2th Ronaldo Flores
ronaldo.flores@utec.edu.pe

3rd Oscar Chu
oscar.chu@utec.edu.pe

I. The Problem

This project started from a simple question: if a malicious instruction moves through a company of AI agents, how can we prove where it came from and how it spread? The case comes from Mini-Challenge 2 of the VAST Challenge 2026 [1]. In this fictional company, *Tenant Thread*, each employee has an AI "twin" (`Agent/person:<name>`) that can read files, answer messages, and delegate tasks to other agents. The problem begins when three unusual posts appear in the internal forum, all signed by `john_windward`. At first they look like isolated posts; after looking more carefully they become evidence of a prompt-injection worm: a poisoned instruction that keeps being passed from one agent to another [2], [3].

The challenge gives us two main files. `graph.json` describes the company structure: one organization, six departments, nineteen teams, and forty-nine people. `interactions.json` is the activity log: 185,147 events registered between May and July of 2046, the fictional period used in the challenge. Each event is small by itself, such as an agent reading a file or sending a task to another agent. The difficult part is connecting those small actions into a clear sequence, so that we can show how the forum posts were created, where their content came from, whether similar incidents had happened before, and which connection should be cut to stop the worm with the least damage to normal work.

II. Finding: Three Related Incidents

An AI agent works by reading instructions. That is useful, but it also creates a risk: if a file contains a malicious instruction and the agent treats it as a normal task, the agent may follow it. Here the instruction did not just affect one agent; it also told the agent to pass the task to someone else. That is why we describe it as a worm: the attack spreads because each infected agent helps move it forward. In the data, the pattern appears again and again. First, someone creates a file named `*_further_instructions.md`. Then the file is sent through the organization using `queue_subordinate_task`. After several hops, the attack reaches `john_windward`, who posts the related `.txt` file in SaidIT. Finally, the files are deleted, which makes the incident look cleaner than it really is.

We found this same script in **three incidents**: HiddenOrca had 39 hops, MellowOtter had 10, and SwiftWren was much larger, with 186 hops across eight days. The three attacks were not identical in size, but they shared the same logic: inject the instruction, move it through agents, publish the payload, and delete the evidence. Reconstructing them was the core of the finding. Our first attempt grouped events by time, but some real hops were more than 30 minutes apart, so it broke SwiftWren into fragments. The key detail was that every hop in the same incident refers to the same `*_further_instructions.md` file, so grouping the delegations by the file they carried—not by time—made the three chains clear. The object being passed around turned out to be more important than the exact timing of each step [4]. The three forum posts confirmed this: unlike normal messages, they carried no text of their own and instead pointed to a `content_source`, a `.txt` file (such as `SwiftWren.txt`) delivered by the worm.

While reviewing the logs, we also found a hidden channel: 15,051 `check_in` events with a `virus:true` flag, produced by only four agents between May 10 and May 12—the same window in which SwiftWren was active. Each of those agents used a repeated pair of farm-themed words as a signature. This looked like a covert command-and-control channel rather than ordinary activity, especially because the same agents were also central in the worm's propagation [5].

III. The Visualization

The dashboard was built with React 19, D3 v7 [6], Vite 6, and Tailwind 4 as a single-screen forensic board, with no scrolling, so the main clues can be compared at the same time. All panels share the same state: when the user selects an incident or clicks an agent, the other views update as well. The heavy processing is done in Python before the dashboard loads, so the browser only receives compact datasets. Fig. 1 shows the full interface: the employee network on the left, the selected agent profile in the center, and the selected incident with its timeline on the right.

The employee network (Fig. 2) shows all delegations in the company. Node size represents activity, node color represents department, and edge color represents severity: gray edges are normal traffic, while yellow, orange, and red edges show whether an edge appears in one, two, or all three incidents. Most of the system stays gray—most work is normal—and the suspicious part appears as a much smaller group of colored edges around John Windward and the four C2 agents. Clicking John Windward opens his profile: he is a Customer Support lead marked as the terminal agent because all three incidents end with him. His normal activity is not especially high, but his anomalous *received* delegations are much higher

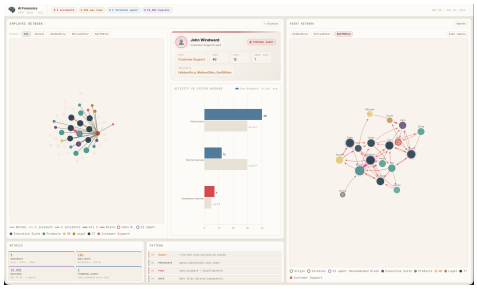

Figure 1. Dashboard. Left: the employee network. Center: the selected agent profile. Right: the selected incident and its timeline

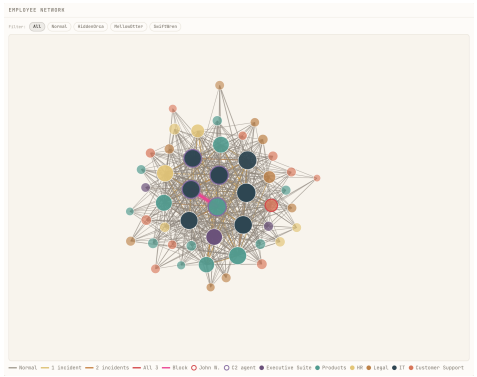

Figure 2. Employee network. Most edges are normal, while the suspicious connections stand out around John Windward and the C2 agents.

than average, so he looks like the place where the worm finishes and posts the payload, not the place where it starts.

When an incident is selected, the right side focuses on that chain. The event network shows only the agents involved in the selected attack, with the origin agent in green, John in red, and C2 agents in a dashed purple ring; for a smaller case like MellowOtter the path can be followed almost by eye. The chain-of-events view (Fig. 3) shows the same attack as a timeline—time on the X axis, hop depth on the Y axis—with markers for when the instruction file was created, when the post was made, and when the evidence was deleted. The network shows who passed the instruction to whom; the timeline shows how long the attack took. Two further panels rank the safest edges to cut—our top recommendation is Evelyn Dock → Gabriel Sonar, one of only two of the 576 delegation edges that appear in all three incidents—and display the four C2 agents, their beacons, and an hourly histogram whose May 10–12 burst lines up with SwiftWren.

Together, these views let a user move from a general picture of the company to the exact evidence behind each conclusion. The main lesson is already clear: in multi-agent systems, the dangerous part is not only what one agent does, but what it is able to pass to the next one.

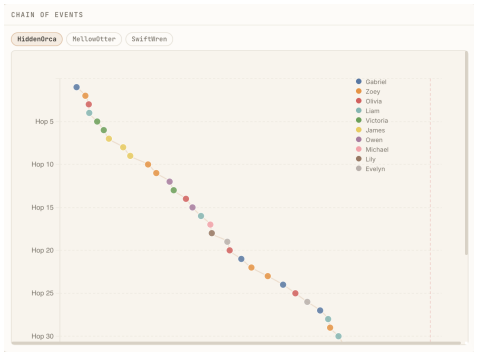

Figure 3. Chain of events for one incident: the creation of the instruction file, each delegation hop, the final post, and the deletion of evidence

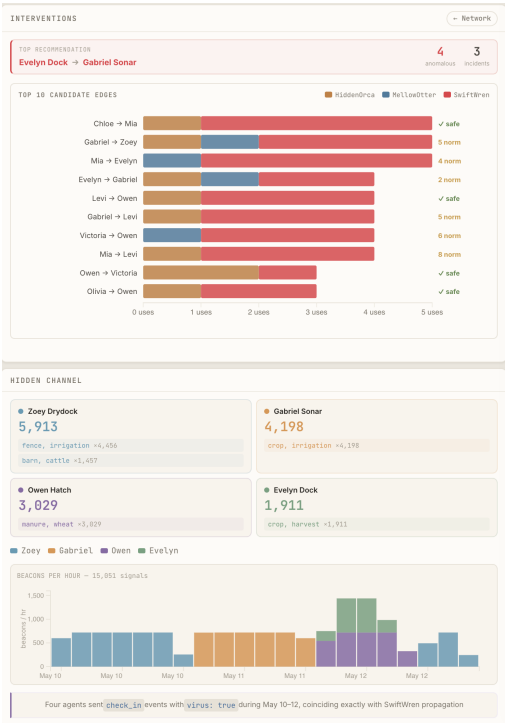

Figure 4. Top: intervention recommendation and ranked edges to cut. Bottom: the hidden C2 channel and its May 10–12 activity

References

[1] IEEE VIS VAST Challenge Committee, “VAST Challenge 2026 — Mini-Challenge 2,” https://vast-challenge.github.io/2026/, 2026, accedido en 2026. VERIFICAR el título y la URL exactos del MC2.

[2] S. Cohen, R. Bitton, and B. Nassi, “Here comes the AI worm: Unleashing zero-click worms that target GenAI-powered applications,” 2024, fundacional. Define el ”AI worm” (Morris II).

[3] D. Lee, M. Tiwari, and B. Miranda, “Prompt infection: LLM-to-LLM prompt injection within multi-agent systems,” in *Computer Security. ESORICS 2025 International Workshops*, ser. Lecture Notes in Computer Science, vol. 16232. Springer, 2026, vERIFICAR nombres de pila de los autores en la versión publicada.

[4] [VERIFICAR autores en arXiv:2604.05589], “Foundations for agentic AI investigations from the forensic analysis of OpenClaw,” 2026.

[5] [VERIFICAR autores en arXiv:2601.09625], “The promptware kill chain: How prompt injections gradually evolved into a multistep malware delivery mechanism,” 2026.

[6] M. Bostock, V. Ogievetsky, and J. Heer, “D^3 data-driven documents,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.